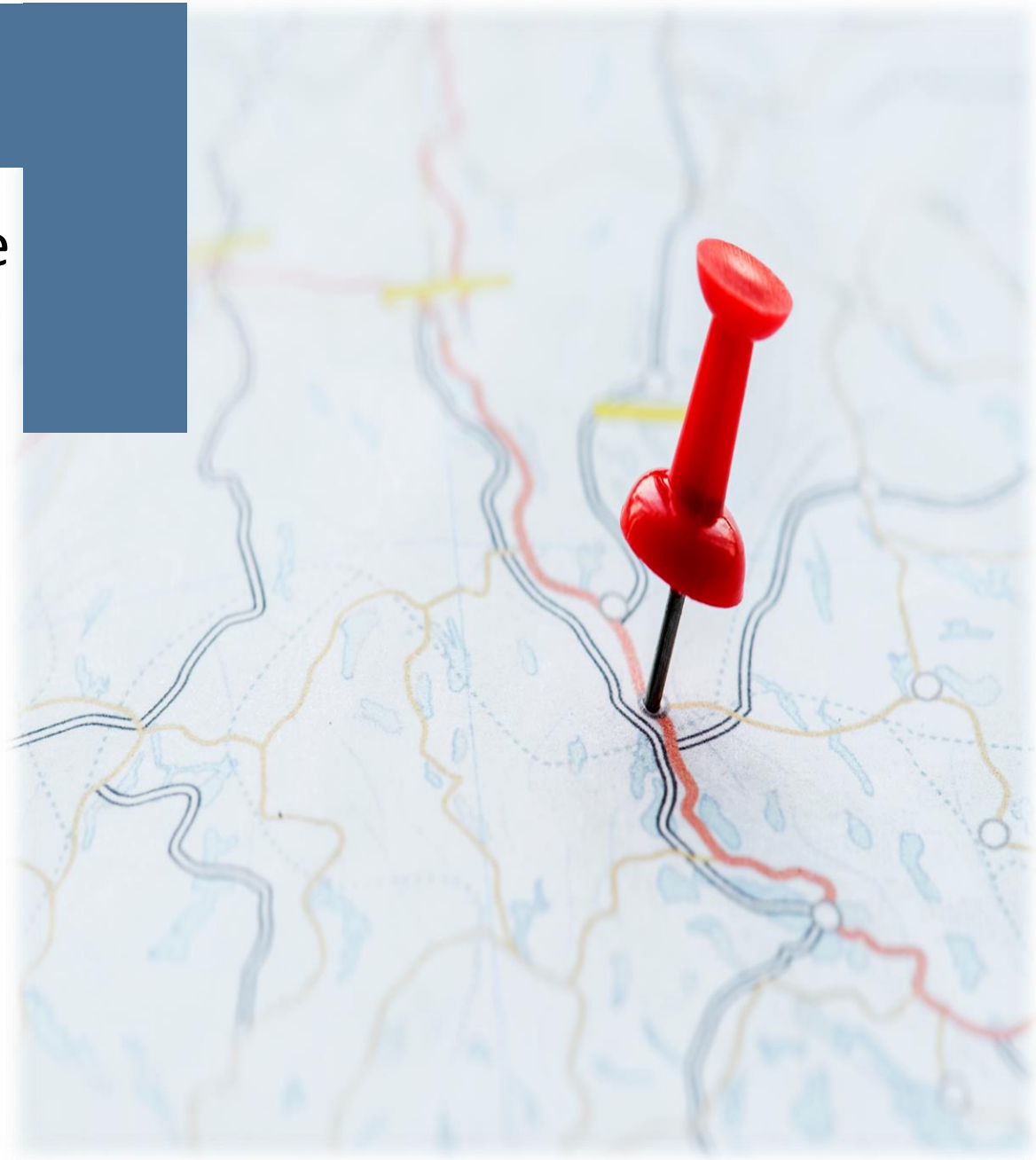
Security Testing

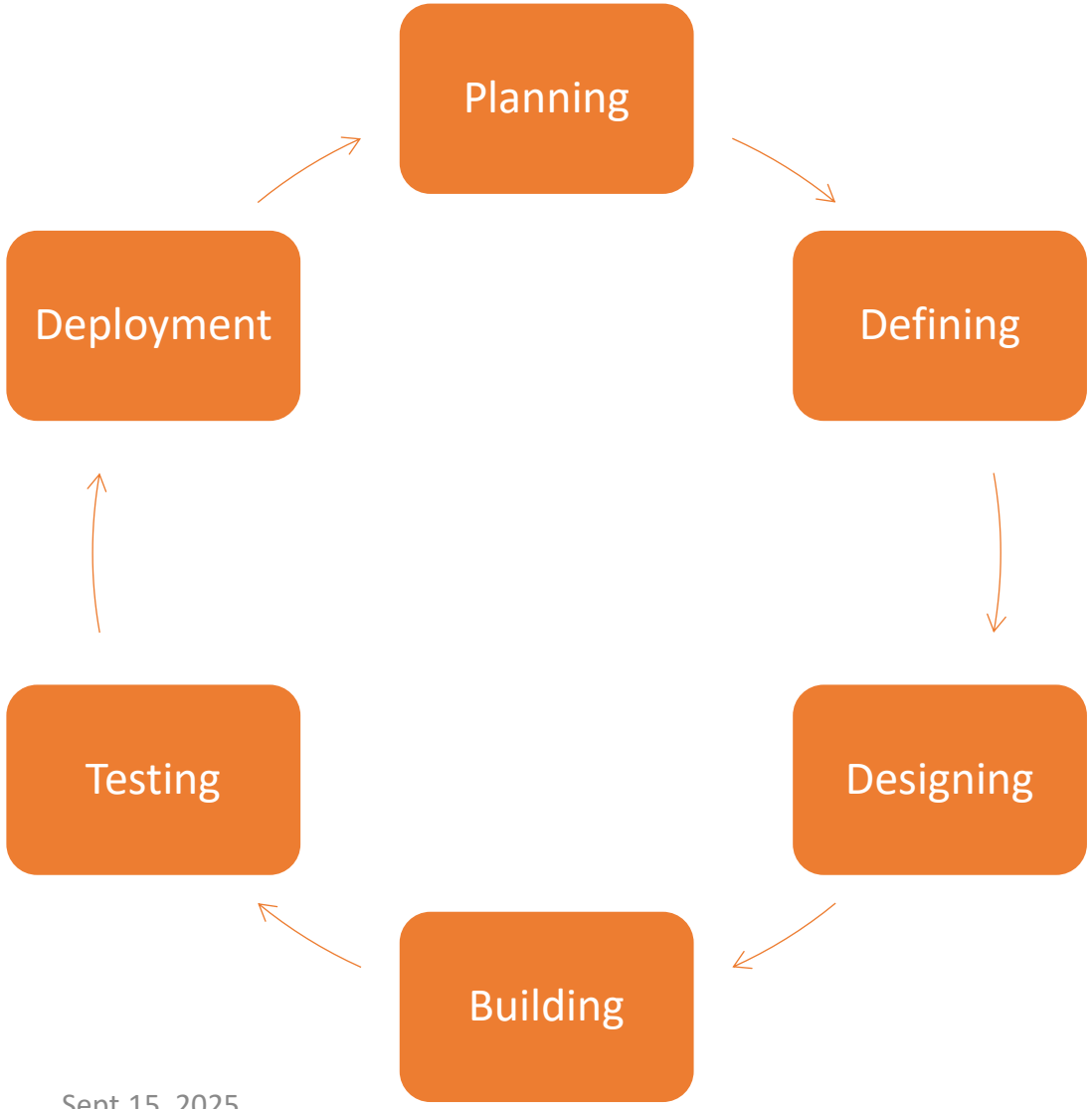
Software Development

Outline

- Secure Software Development Life Cycle
- DevSecOps

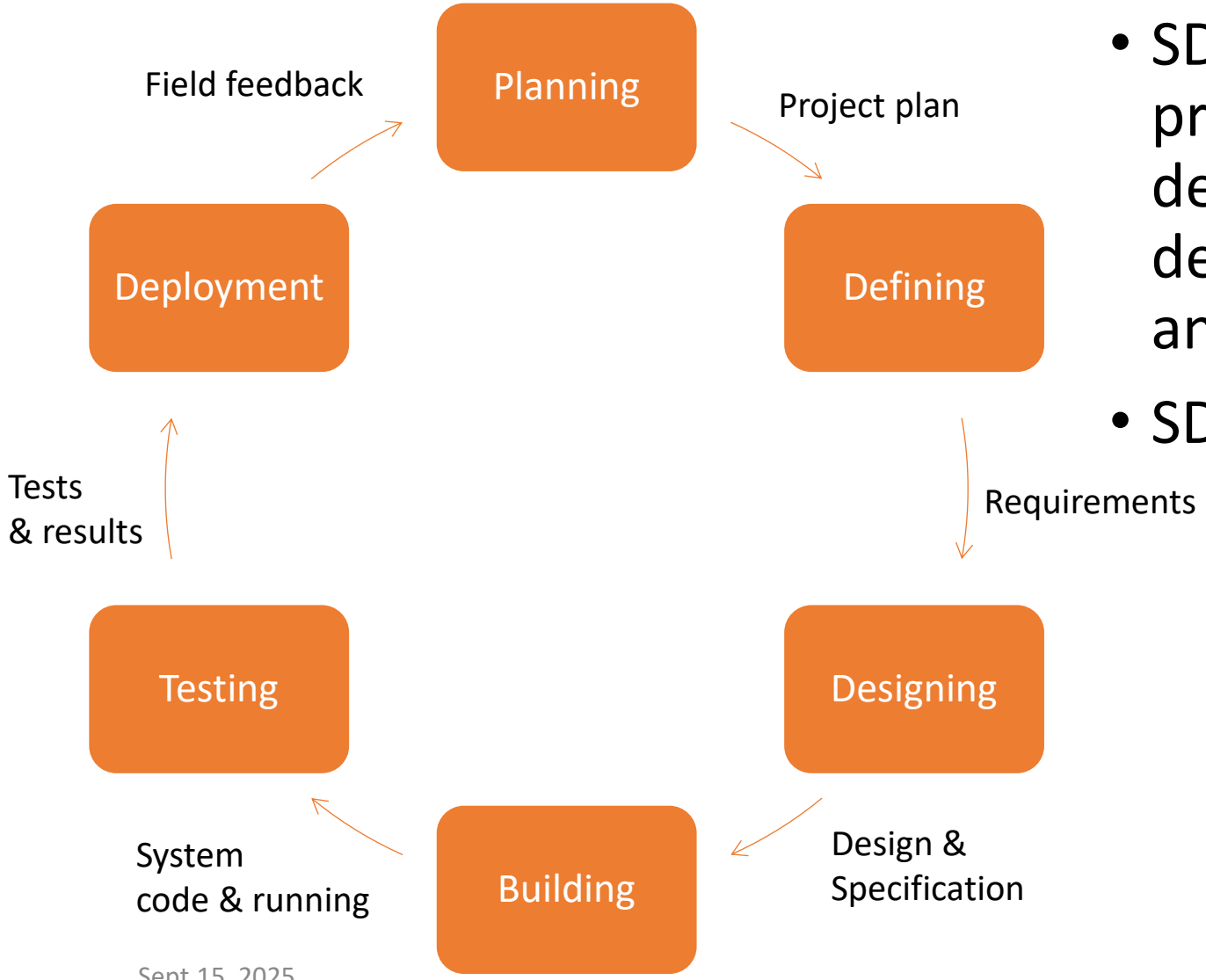


System Development Life Cycle



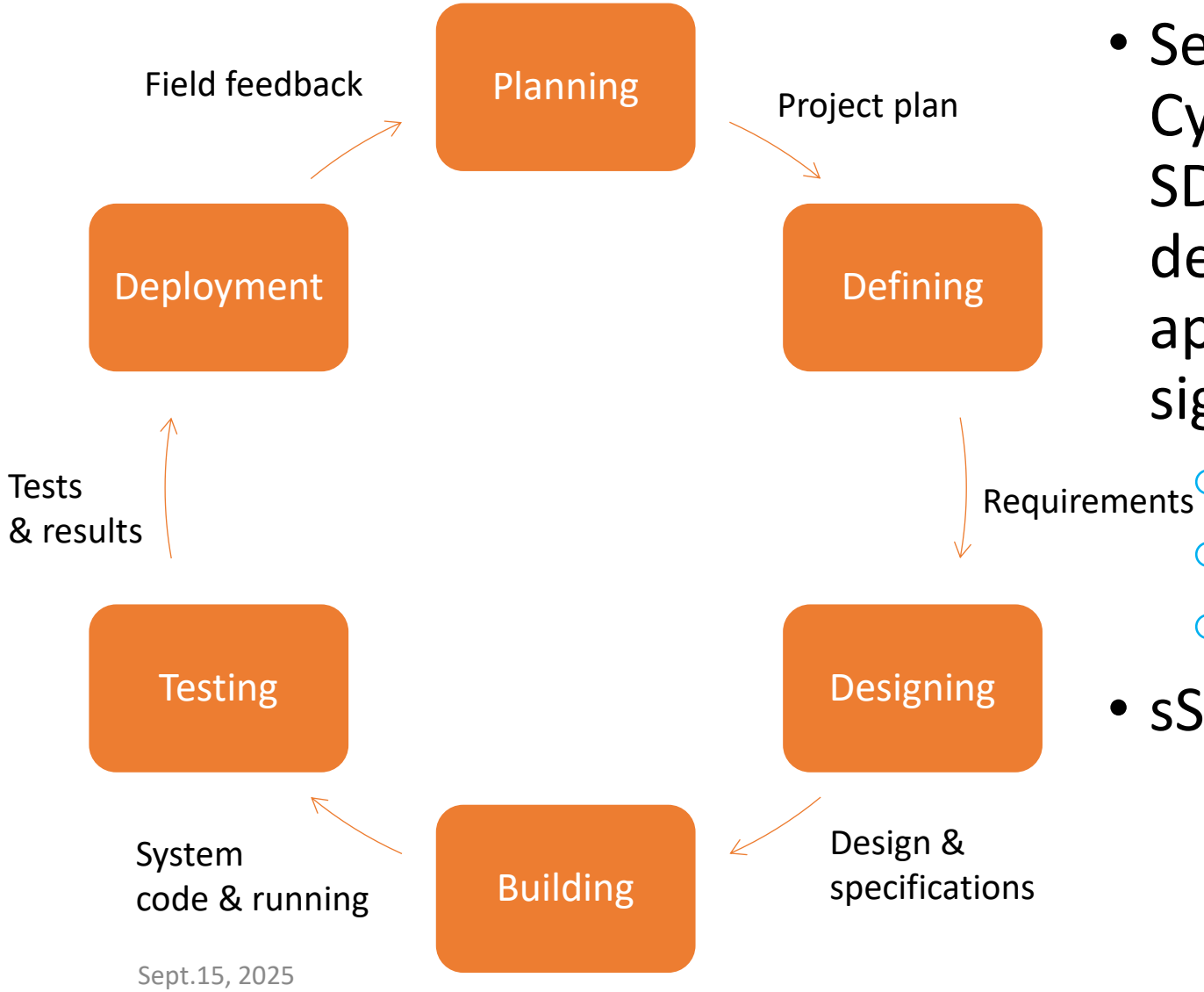
- SDLC is the series of processes and procedures, in the software development cycle, to enable development teams create software and applications
- SDLC is divided in phases

System Development Life Cycle: artifacts



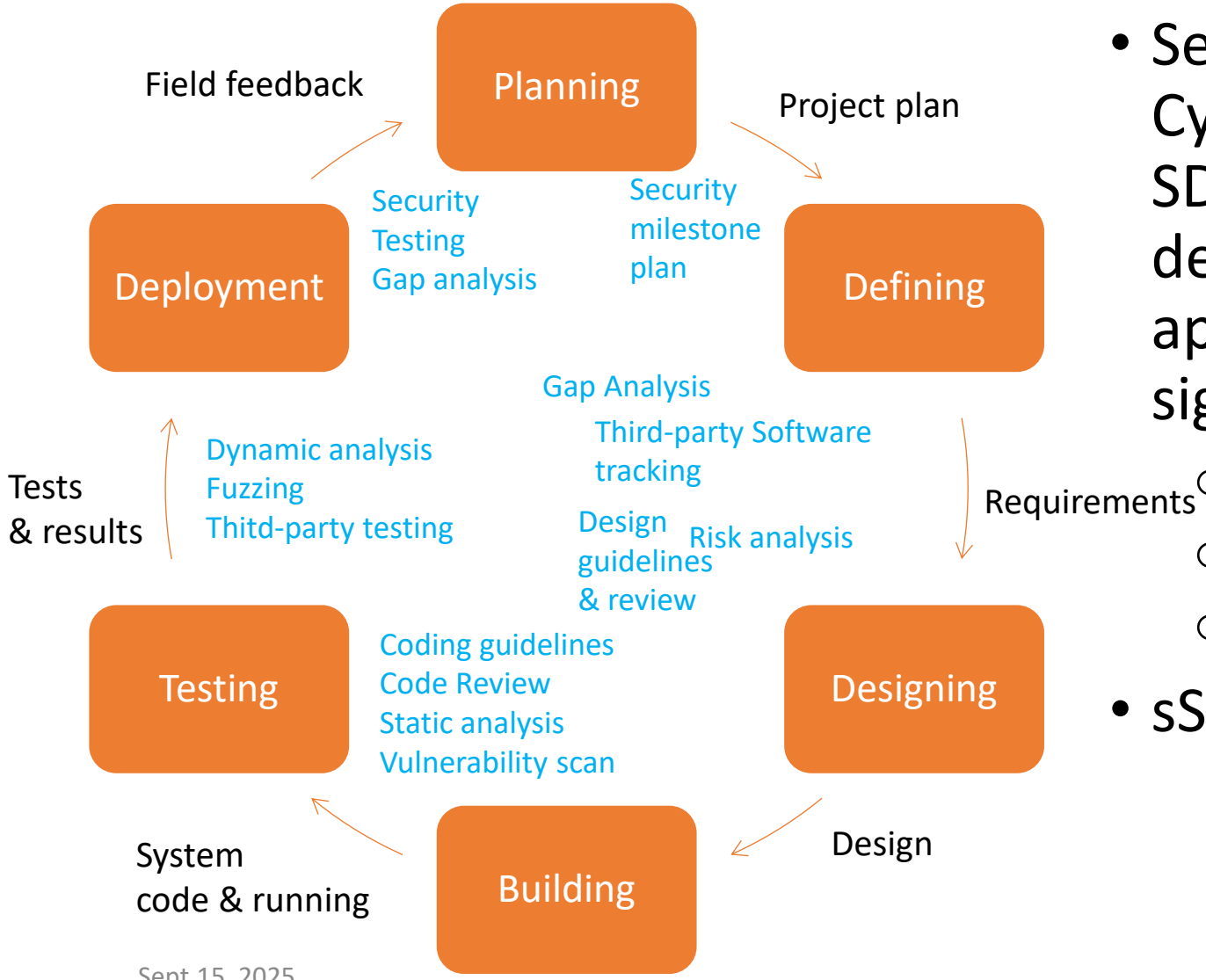
- SDLC is the series of processes and procedures, in the software development cycle, to enable development teams create software and applications
- SDLC is divided in phases

Secure SDLC



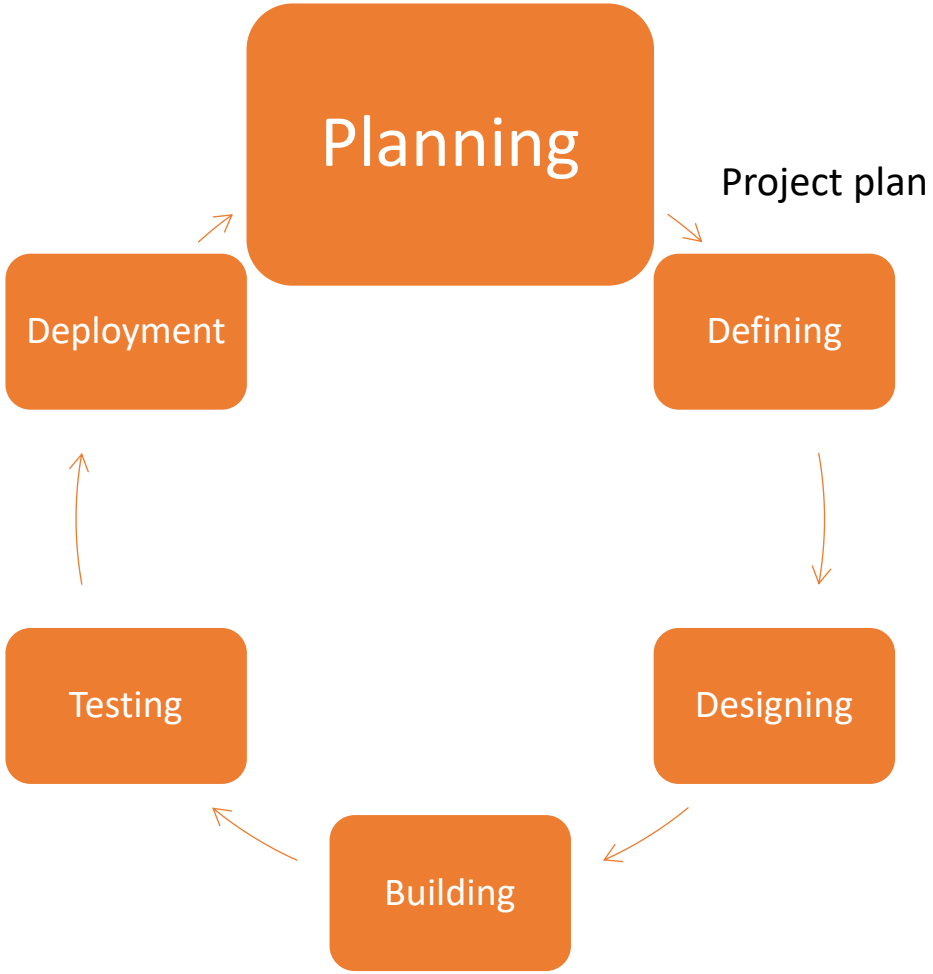
- Secure System Development Life Cycle is defined, like the traditional SDLC, but it focuses on the development of software and applications in a manner that significantly:
  - reduces security risks
  - eliminate security vulnerabilities
  - reducing costs
- sSDLC is also divided phases.

Secure SDLC: techniques by phase



- Secure System Development Life Cycle is defined, like the traditional SDLC, but it focuses on the development of software and applications in a manner that significantly:
  - reduces security risks
  - eliminate security vulnerabilities
  - reducing costs
- sSDLC is also divided phases.

SDLC: Planning

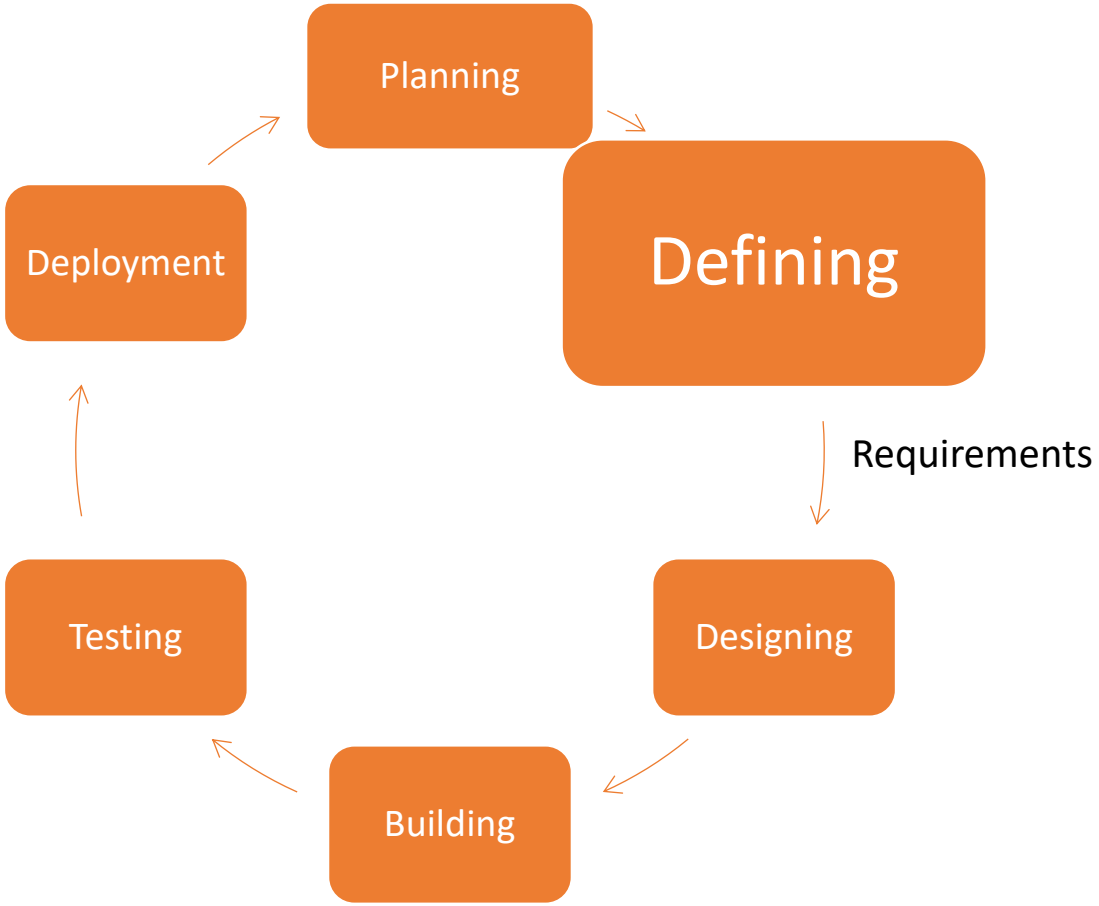


- Performed by the senior members of the team with inputs from:
  - the customer
  - the sales department
  - market surveys
  - domain experts in the industry
- Plan the basic project approach
  - economical perspective
  - operational perspective
  - technical perspective

Planning: security perspective

- Goals:
  - define the security objectives required by the software
  - identify what are the possible threats
  - Identify what regulations the organization needs to follow
- Plan the output of the development team :
  - security milestones
  - required certifications
  - risk assessments
  - key security resources
  - required third-party resources

SDLC: Requirements



- Define document the product requirements
- Get them approved from the customer or the market analysts
- SRS (Software Requirement Specification)
  - ISO/IEC/IEEE 29148:2018
  - “FACTS”:
    - Functional Requirements
    - Analysis Model
    - Cognitive Model
    - The Content & Structure of the Specification
    - Specification guidelineas

Requirements: security perspective

Three main activities:

(1) Gap Analysis

(2) Risk Assessment

(3) Third-Party Software Tracking

Requirements: security perspective (1)

(1) Gap Analysis

- Security training sessions for developers, designers, architects, and QA
  - Focus on secure design principles, security issues, web security or encryption
- Security awareness sessions
  - Not only for developers but they have to involve everyone that is connected to the project
  - Technical-level sessions
  - Inclusion of any topics related to cybersecurity threats and risk impact and management

Requirements: security perspective (1)

(2) Risk Assessment

- Risk (def.)
  - It is the potential that a given threat will exploit vulnerabilities of an asset or group of assets and thereby have an impact on the identified assets
  - An asset is a relevant (piece of) information, data or service or product
- Risk Assessment (def.)
  - the process to identify and determine risks that affect organization's operations, assets, individuals, other organizations and even the nation
  - different assessment methods can be based on different principles
    - Threat Based
    - Start from what can go wrong and defend from it
    - Asset Based
    - Start from what is worth and protect it
    - ….

Requirements: *summary* of key notions

Vulnerability: it is a weakness or bug in a system, process, or activity. It is an internal condition that, if exploited, could cause an issue.

- E.g., a software that implements a known CWE
- E.g., an outdated software with known security bugs
- E.g., an employee not trained in security policies

Threat: it is a potential external danger that could exploit a vulnerability to damage an asset. It is the actor or event that *could* cause the issue.

- E.g., a hacker trying to infiltrate a network
- E.g., malware (viruses, ransomware) spreading
- E.g., an unintentional human error (e.g., click on a phishing link)

Risk: it is the probability that a threat will exploit a vulnerability, combined with the resulting impact (the damage).

- E.g., the risk of a hacker (*threat*) exploiting outdated software (*vulnerability*) to steal customer data (*impact*).

Requirements: Threat Modeling

Main steps (not just software):

1. Clearly identify and characterize the system to analyze and the (host) environment
2. Identify assets (relevant data/service to protect)
3. Identify threats (*cyber attacker or employee or ..*)
4. Identify vulnerability (*misconfig. or old sw or ..*)
5. Estimate impact on asset (*minor, severe, etc.*)
6. Estimate probability of event (*unlikely, likely, etc.*)
7. Rank threats by risk *(low, medium, high)*

   Risk ≈ Impact * Likelihood
8. Decide which threats to respond to (thresholds)
9. Decide mitigation and control actions for the threats
   - which techniques and technologies use and when

	Impact				
Likelihood	1 No impact	2. Minor	3. Severe	4. Critical	5. Catastrophic
5. Certain					
4. Very likely					
3. Likely					
2. Unlikely					
1. Very Unlikely					

To get insights:
Cybersecurity Risk Assessment course - UNITN

Requirements: Threat Modeling

Main steps (not just software):

1. Clearly identify and characterize the system to analyze and the (host) environment
2. Identify assets (relevant data/service to protect)
3. Identify threats (*cyber attacker or employee or ..*)
4. Identify vulnerability (*misconfig. or old sw or ..*)
5. Estimate impact on asset (*minor, severe, etc.*)
6. Estimate probability of event (*unlikely, likely, etc.*)
7. Rank threats by risk *(low, medium, high)*

   Risk ≈ Impact * Likelihood
8. Decide which threats to respond to (thresholds)
9. Decide mitigation and control actions for the threats
   - which techniques and technologies use and when

- Env: ecommerce app and its system
- Asset: provided service and sensitive data
- Identify threats: DoS & SQL injection
- Vulnerability: bugs in the application code (e.g., lack of input controls)
- Estimate impact on asset: **severe** (business down) & **critical** (data breach)
- Probability of event: **likely** (frequent attack)
- Risk: **medium** (orange) & **high** (red)
- Mitigate? **Yes**
- Control actions: use filtering in route & code analysis to discover and fix the bugs

	Impact				
Likelihood	1 No impact	2. Minor	3. Severe	4. Critical	5. Catastrophic
5. Certain					
4. Very likely					
3. Likely					
2. Unlikely					
1. Very Unlikely					

To get insights:
Cybersecurity Risk Assessment course - UNITN

Example of Threats, properties, and mitigations

- Spoofing Identity → authentication
  - authentication, protect keys & passwords, ...
- Tampering with Data → integrity
  - access control, hashes, digital signatures, MACs (message authentication codes), write-once storage...
- Information Disclosure → confidentiality
  - access control, encryption, not storing secrets, ...
- Denial of Service → availability
  - graceful degradation, filtering, increase server resources
- Elevation of Privilege → access control
  - access control, sandboxing, ...

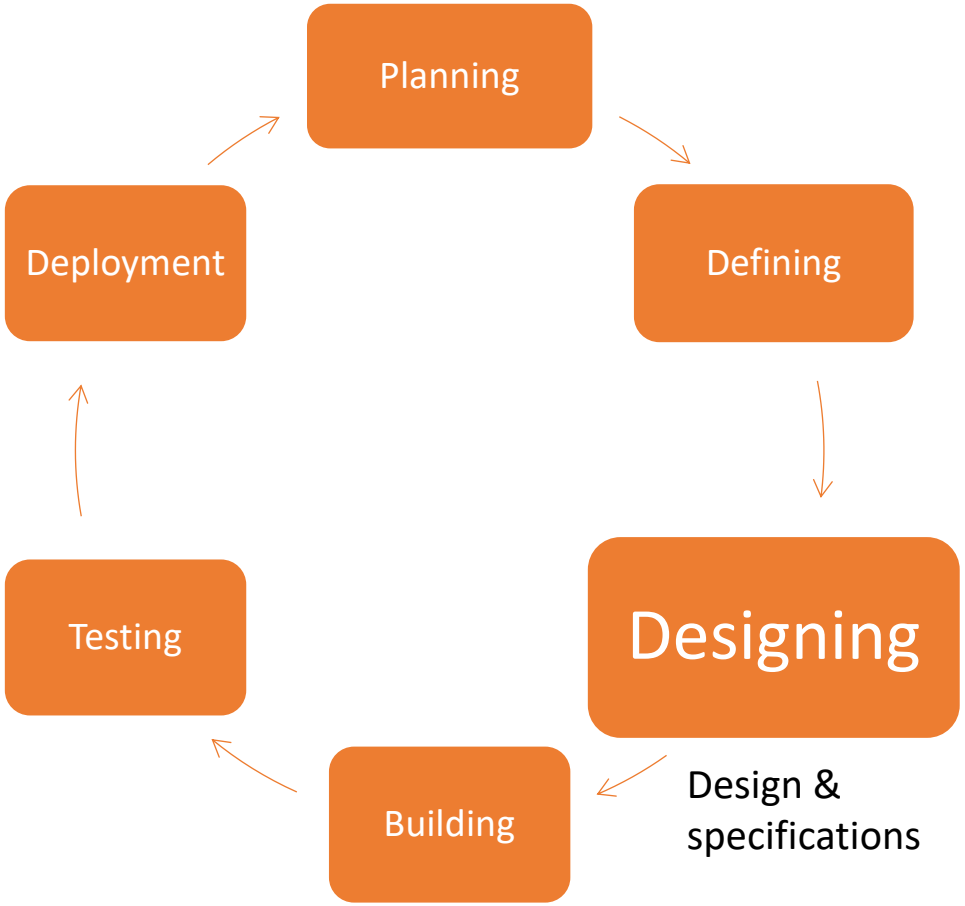
...

Requirements: security perspective (5)

(3) Third-Party Software Tracking

- The list of both open-source and commercial third-party software (e.g., libraries, frameworks) must be defined
- This list allows to check the used third-party software
  - E.g., updates and patches (due to security issues) can be early signalized

SDLC: Design and product architecture



- Architects come out with the architecture for the product
- DDS - Design Document Specification defines design based on the following factors:
  - risk assessment
  - product robustness
  - design modularity
  - budget
  - time constraints
- Architecture
  - of each project module
  - of the whole project
- Data flow representation with the external and third-party modules (if any)

Architecture: security perspective

Relevant rules (guidelines):

- Modularity: divide the software in small parts (components, modules, functions) with interfaces that enable communication
- Isolation: each part should work correctly even if others fail (e.g., return wrong results, send invalid requests and/or invalid arguments)
- Reuse: modularity and isolation push toward the reuse of each part
- Hierarchical organization of parts (modules and functions)
- Simplicity (complex → difficult to verify, test and maintain)

Design: security perspective

Relevant rules (guidelines):

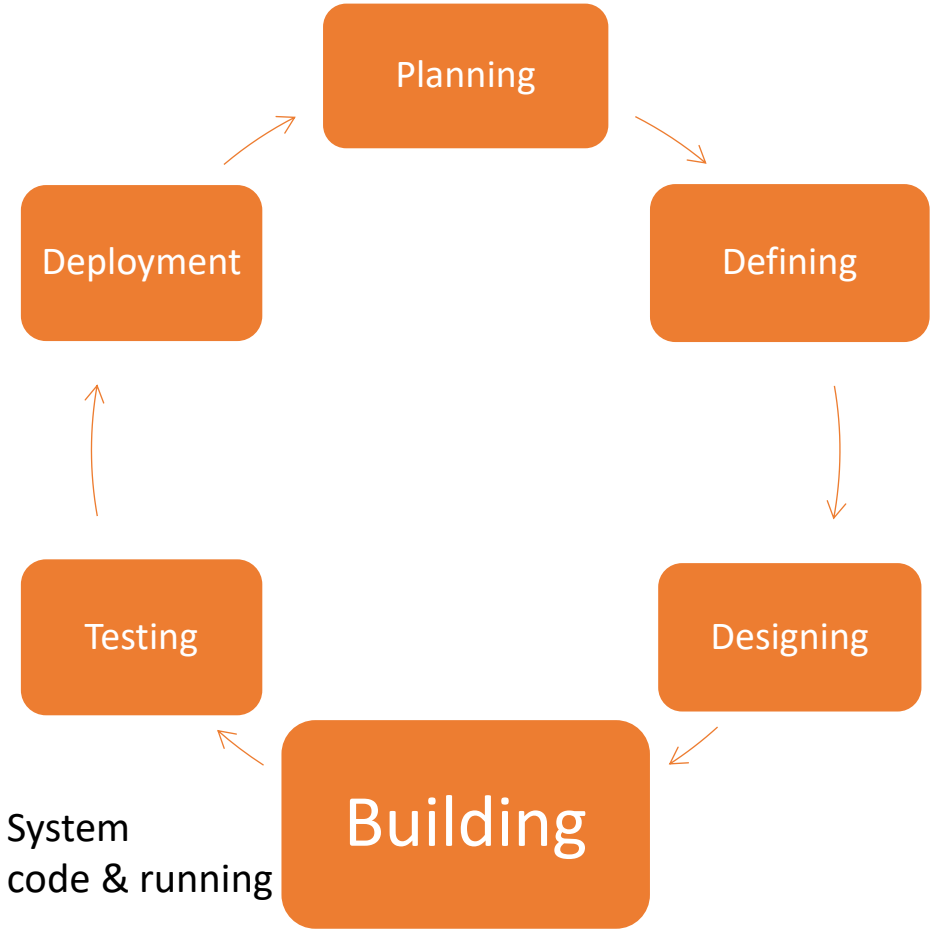
- Make security-sensitive parts of your code small
- Least-privilege principle
  - program should run on the least privileged account possible
  - same for accessing databases, files etc.
  - revoke a privilege when it is not needed anymore
- Choose safe defaults
- Deny by default
- Limit resource consumption
- Fail gracefully and securely
- Question again your assumptions, decisions, etc.

Design: security perspective

Design Review

- The review activity of the developer on the functional feature design must include a review of security aspects
- In the security review, the developer has to act as an attacker to discover the feature vulnerabilities
- When reviewing the design, the developer needs to know the most common design security pitfalls.
- Often, during the review, the developer use a list checklist for secure design, e.g., that ensures that important security events are logged, failures are managed.

SDLC: Implementation



- The programming code is generated as per DDS
- if the design is performed in a detailed and organized manner, coding is not difficult
- Different programming languages such as C, C++, Pascal, Java and PHP are used for coding.
- The programming language is chosen with respect to the type of software being developed

Implementation and bugs: security perspective

- **Bug**: the system isn't behaving as it's designed to behave
- **Vulnerability**: is a chain of weaknesses linked by causality: it starts with a bug and ends with a final error, which if exploited leads to a security failure.

- Many security issues are related to vulnerabilities due to bugs, defects, or design flaws in a targeted software system

- Software bugs, defects, or design flaws can be exploited to affect an undesired behavior

What to do?

- Vulnerabilities exist and can be exploited

- What we have to do?

  - (A) Follow security guidelines when programming
  - Think of security implications
  - Reuse trusted code (libraries, modules etc.)
  - Write good-quality, readable and maintainable code
  - (B) Review the code
  - (C) Verify and test the code and the system

A) Guidelines: towards Secure programming

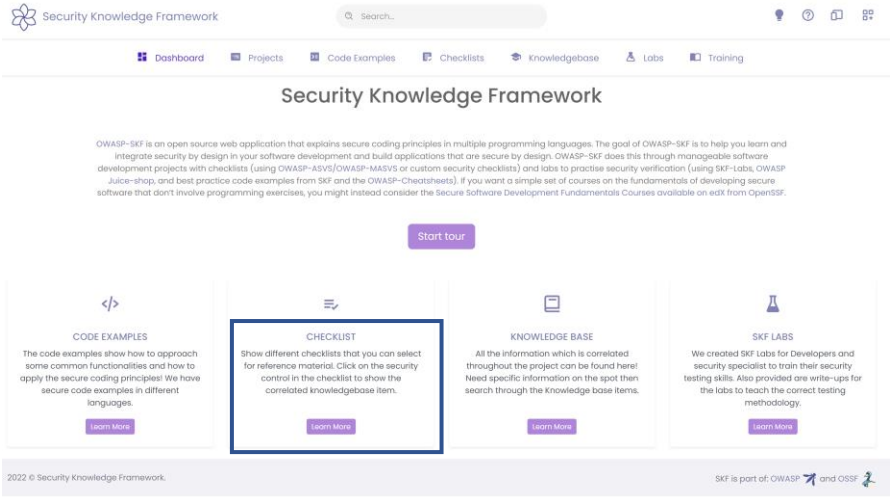
- Input data cannot be trusted
- Default-deny rule: allow only “good” characters and formulas and reject others (instead of looking for “bad” ones)
- Input and memory validation and checking
- Separate data from code
- Check the Command-line arguments
- Environment: avoid any assumptions about the environment (variables, PATHs, etc.)
- Signals: catch and control signals
- Protect passwords and secret information: don’t hard-code it, use external files instead (possibly encrypted), use certificates
- Deal with errors and exceptions
- Configuration files: if accessible by untrusted users
- Be careful when handling (create/mange) files, even if temporary files, avoid them whenever possible
- Beware of race conditions
- Do not trust your client

A) Guidelines: framework

Frameworks can help in the secure coding rule knowledge and use

E.g., OWASP Security Knowledge Framework (SKF)

- Open-source application that explains secure coding principles in multiple programming languages
- It is based on checklists and labs to practice security
- The goal of is to support developers in understanding and using security by design



Additional reading
https://owasp.org/www-project-security-knowledge-framework
https://demo.securityknowledgeframework.org/dashboard (demo)
https://github.com/blabla1337/skf-flask

B) Code Review

- Large amount of security problems are caused by simple bugs that can be detected in code
  - E.g., buffer overflow, use of unsafe functions, etc.
- Code review
  - It is conducted by a developer that manually checks the code
  - It focuses on looking for implementation bugs
  - the developer needs to know the most common code security pitfalls (to be able to find it)
  - Often, a checklist is used for checking secure coding, e.g., that ensures the authentication and the user-input validation
- Code review is boring, difficult and tedious

C) Security Testing: overall view

Security testing checks how vulnerable is the software.

Static Analysis

- identifies the location of weaknesses by analyzing the software without executing it

Dynamic Analysis

- identifies weaknesses by running the software, helping find infrastructure flaws and patch errors

Vulnerability Scanning

- injects malicious inputs against running software to check how the program reacts

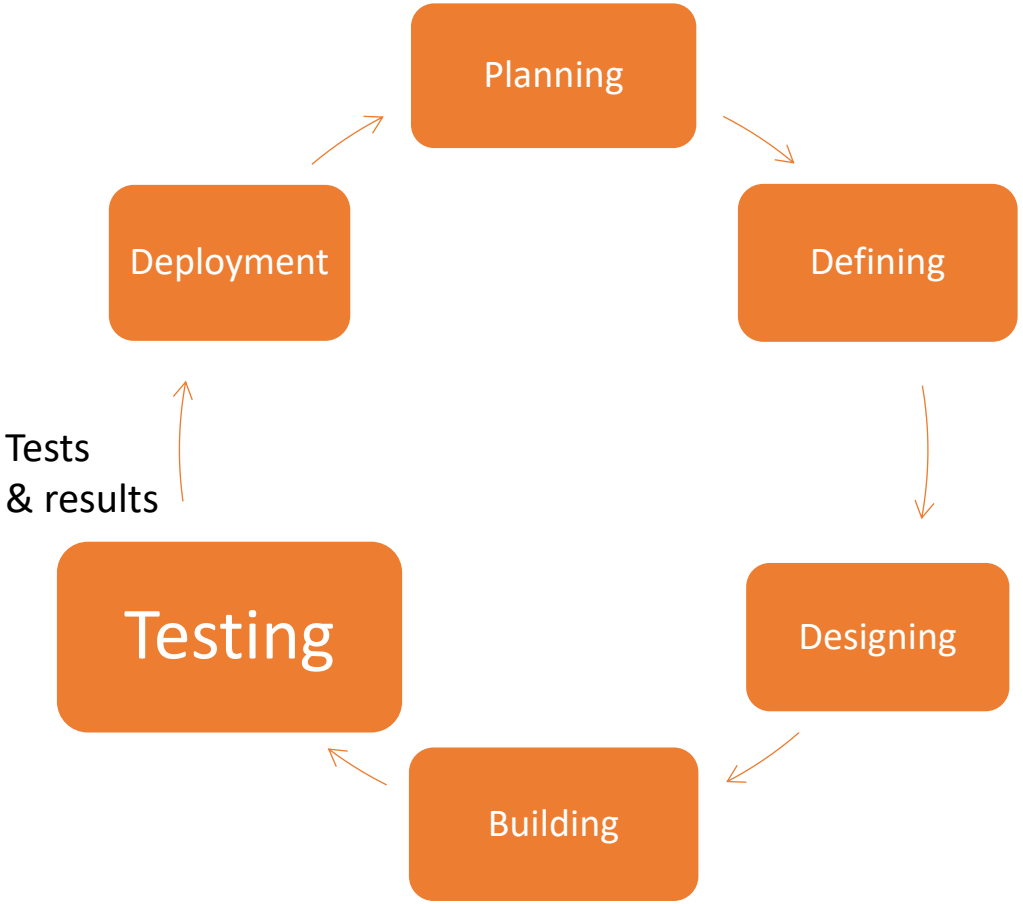
Fuzzing

- involves giving invalid, random data to a program, to check for access protocols and file formats. It helps in finding bugs that humans often miss by generating random input and try possible variants

Third-party (penetration) testing

- It focuses on the testing of reusable components, it simulates attacka to discover coding or system configuration flaws and discover vulnerabilities.

SDLC: Testing



- Testing at every stage of product development is suggested
- Product defects are reported, tracked, fixed and retested
- Reiterated until the quality reaches the one specified in SRC

C) Security Testing: overall view

Security testing checks how vulnerable is the software.

Static Analysis

- identifies the location of weaknesses by analyzing the software without executing it

Dynamic Analysis

- identifies weaknesses by running the software, helping find infrastructure flaws and patch errors

Vulnerability Scanning

- injects malicious inputs against running software to check how the program reacts

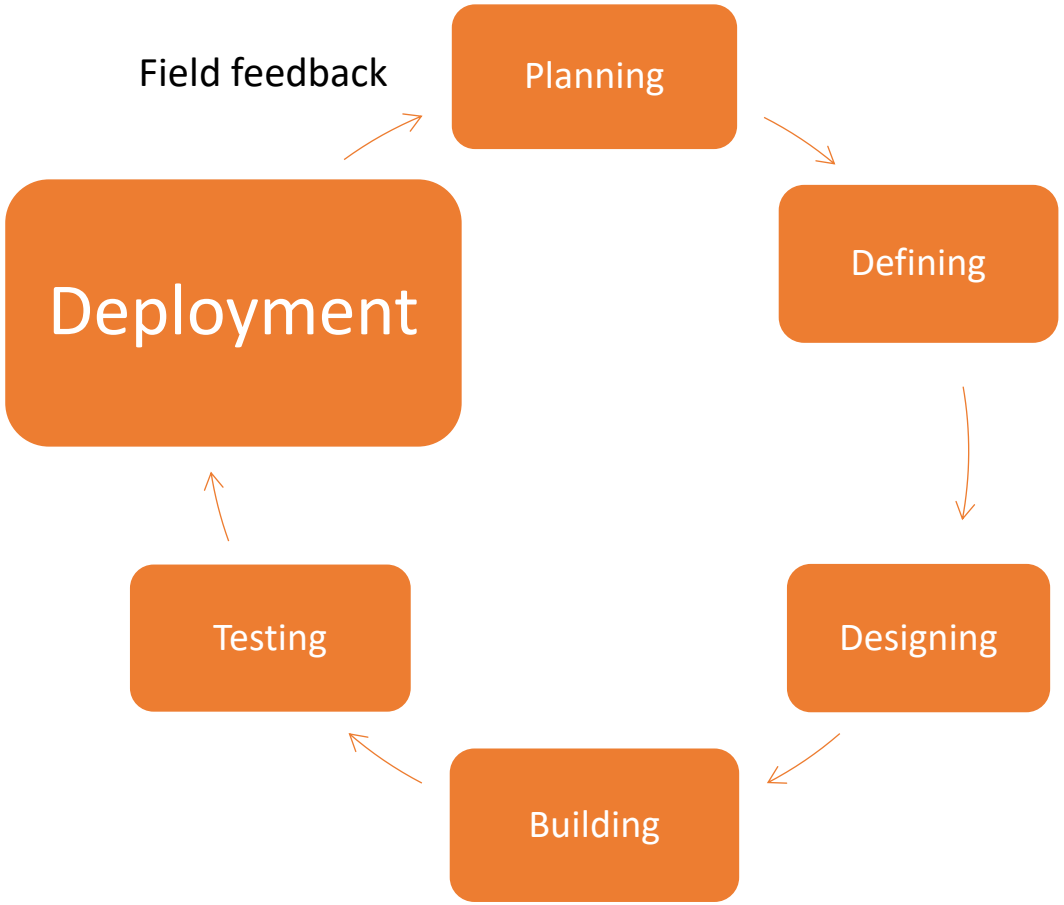
Fuzzing

- involves giving invalid, random data to a program, to check for access protocols and file formats. It helps in finding bugs that humans often miss by generating random input and try possible variants

Third-party (penetration) testing

- It focuses on the testing of reusable components; it simulates attacks to discover coding or system configuration flaws and discover vulnerabilities.

SDLC: Deployment and Maintenance

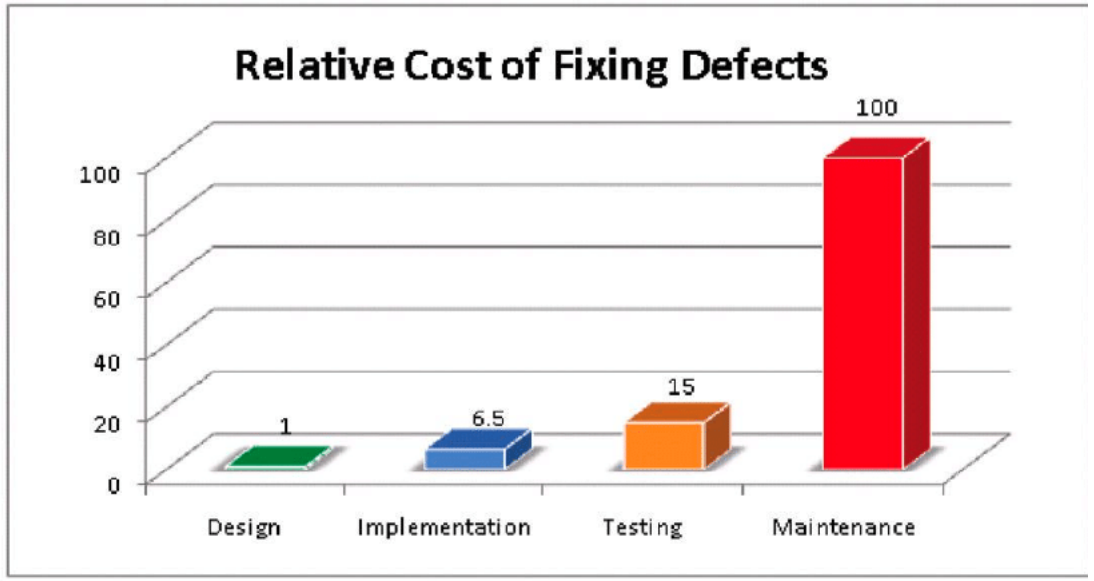


- Formal release
- Alpha/beta releases to collect user feedback on the software use

Deployment and maintenance: security perspective

- Final Gap analysis
- Final security test

SDLC: When to fix a bug (1)



- Bugs should be fixed as soon as possible

→Same for security bugs

Citation by: IBM System Science Institute Relative Cost of Fixing Defects

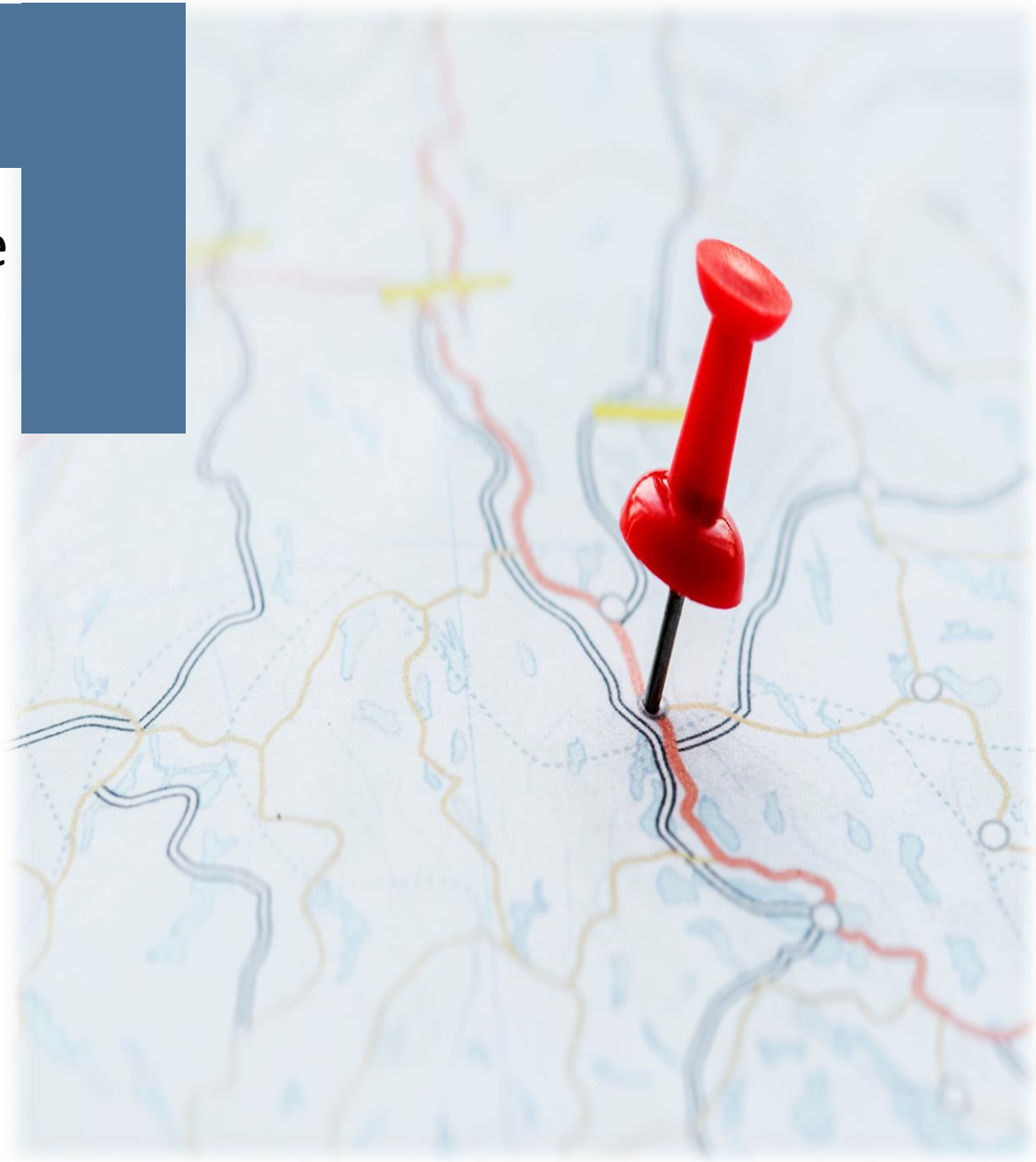
SDLC: When to fix a bug (2)

- Security – in each phase of software development
  - not added after implementation
- Bugs must be fixed as soon as possible
- Security testing is a set of approaches and techniques
- Security testing is related to code guidelines

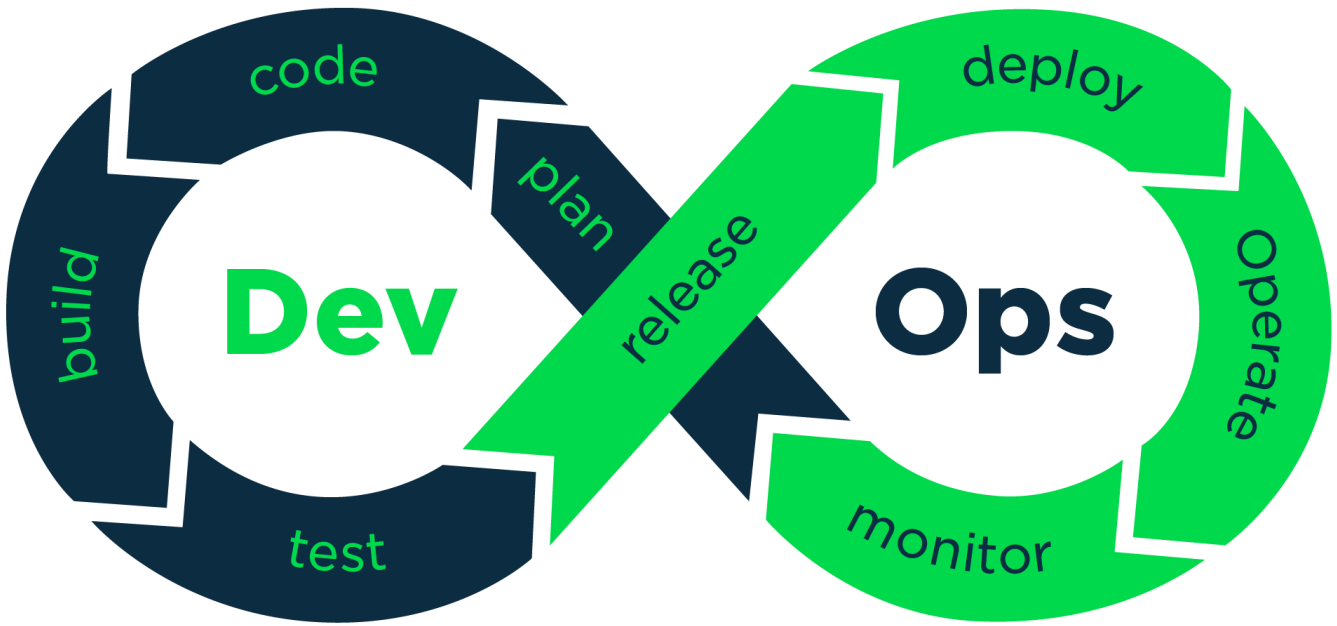
This is not good security!

Outline

- Secure Software Development Life Cycle
- **DevSecOps**



DevOps



- DevOps
  - “Development”
  - and “Operations”
- Based on 8 phases
  - Rapid delivery
  - Quick feedback (aka “fail fast”)
- Example of DevOps methods
  - Scrum
  - Agile

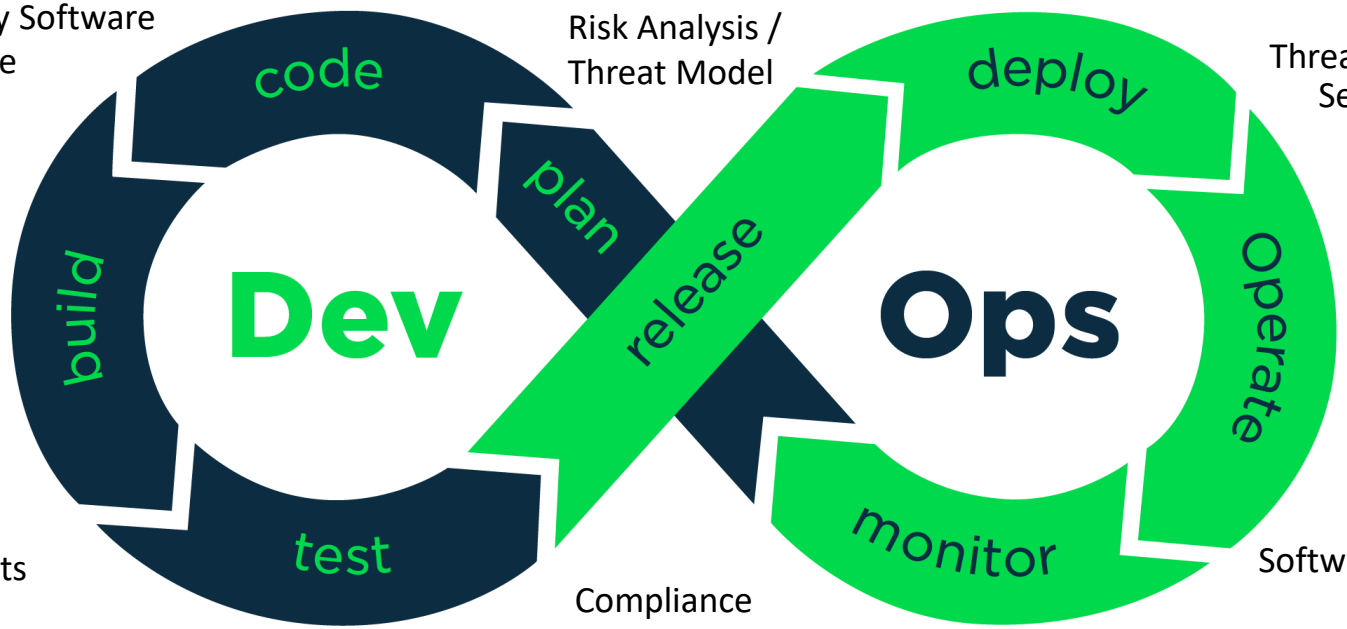
DevOps practices

- Continuous Integration (CI) - developers regularly merge their code changes into a central repository, after which automated builds and tests are run.
  - find and address bugs quicker
  - improve software quality
  - reduce the time it takes to validate and release new software updates

- Continuous Delivery (CD) - code changes are automatically built, tested, and prepared for a release to production.
  - always have a deployment-ready build artifact that has passed through a standardized test process

- Other practices, like microservices, infrastructure as code, monitoring and logging, communication and collaboration.

DevSecOps



Secure coding
Static code analysis

Code Review
Third-party Software governance

Risk Analysis / Threat Model

Configuration validation

Threat Intelligence
Security scan

Secure Testing
- Static code analysis
- Dynamic analysis

Intrusion detection

Response

Secure Testing
- Dynamic analysis
- Interactive analysis
- Pentests / Red Teams

Pentests

Fuzzing

Compliance Validation

Monitoring

Software restore/recovery

Roles

- **Security testing**: test the system for discovering as much as possible technical weaknesses and vulnerabilities. The goal is to identify and evaluate the vulnerabilities. Static analysis and vulnerability assessment are part of security testing.
- **Penetration testing**: test the system for discovering as much as possible technical vulnerabilities of the system. The goal is to exploit the system vulnerabilities to show the possible impact. Typically, vulnerability assessment is not part of penetration testing.
- **Red team**: ethical hackers that attack the system, by playing the role of hackers, for measuring the effectiveness of the people, processes and technologies used to defend the system. The goal is not only to discover and exploit vulnerabilities, but evaluate the system's defense capability.
- **Blue team**: the security team of the system that defend the system during and after an attack